

# NLP Project: Information Retrieval System and Retrieval Extensions

Author 1<sup>1</sup>, Author 2<sup>1</sup>, and Author 3<sup>1,2</sup>

<sup>1</sup> Indian Institute of Technology, Madras, Chennai 600036, TN, India  
{roll\_no.1,roll\_no.2,roll\_no.3}@smai1.iitm.ac.in

<sup>2</sup> University 2 email

**Abstract.** This report consolidates the full assignment workflow for our Information Retrieval system using the Cranfield dataset. It includes a manual retrieval example, evaluation of a baseline TF-IDF system, an analysis of Vector Space Model limitations, and an extended project exploring multiple retrieval enhancements including LSA, n-gram matching, Word2Vec retrieval, statistical language modeling, BM25+, neural reranking, and Learning-to-Rank. The report follows the ‘Main.tex’ template structure while preserving the substantive content from our original ‘custom.tex’.

**Keywords:** Keyword 1 · Keyword 2 · Keyword 3

## 1 Introduction

## 2 Problem Statement

## 3 Background

## 4 Approach

**Approach 1: Latent Semantic Analysis (LSA)** To address the limitations of the Vector Space Model, specifically synonymy and polysemy, we implemented Latent Semantic Analysis (LSA). This approach moves away from exact keyword matching by trying to uncovering the underlying topics (semantic structure) of the corpus using singular value decomposition.

**The Hypothesis:** Every document has underlying topics that are not explicitly stated but are reflected in the word co-occurrences across the corpus. The co-occurrences are enough to capture synonymy and semantic context.

**Mathematical Formulation:** The core of LSA is the Singular Value Decomposition (SVD). Given the Term-Document Matrix  $A$  of rank  $r$ , SVD decomposes it into three matrices:

$$A = U\Sigma V^T$$

To reduce noise and capture latent concepts, we approximate  $A$  by keeping only the top  $k$  singular values:

$$A \approx A_k = U_k \Sigma_k V_k^T$$

where  $U_k$  represents the term-concept mapping and  $V_k^T$  represents the document-concept mapping in a  $k$ -dimensional space.

**Implementation Details:** In our implementation, we followed these specific steps to optimize retrieval performance:

1. **Log-TF Weighting:** Instead of raw counts, we used Log-TF weighting ( $1 + \log(\text{freq})$ ). This dampens the effect of very high-frequency words, which is critical for stable SVD decomposition.
2. **Smoothed IDF:** We applied smoothed Inverse Document Frequency to the Term-Document Matrix before decomposition to highlight discriminative terms.
3. **Query Projection:** Unlike the documents which are pre-computed, queries are projected into the latent space using the transformation:

$$q_{latent} = q^T U_k \Sigma_k^{-1}$$

4. **Similarity Measurement:** Retrieval is performed by calculating the Cosine Similarity between the query vector and document vectors within the reduced  $k$ -dimensional latent space.

#### LSA-Synset hybrid approach:

Another approach, which we worked on in the domain of LSA, was to disambiguate each word occurrence using its local context and WordNet, then build the LSA model on semantic concepts.

For each token in a document or query, we use a lightweight Lesk-style algorithm to select the most appropriate WordNet synset:

1. **Candidate Selection:** For a given token, we retrieve all WordNet synsets that include that word, giving preference to noun synsets.
2. **Context Overlap:** We build a context window around the token and compare it against the signature of each candidate synset. The signature includes the synset's definition, examples, and related concepts (hypernyms and hyponyms).
3. **Best Match:** We select the synset with the highest overlap between the local context and its signature, breaking ties by lemma match bonus. If no context is available, we default to the first sense.

Once tokens are mapped to synsets, we build a concept-document matrix and proceed with the standard LSA-based document ranking and retrieval.

**Approach 2: N-gram Based Vector Space Model** To capture local contextual information and meaningful phrases that are often lost in a unigram-based

bag-of-words model, we implemented an N-gram retrieval system (specifically focused on Bigrams,  $n = 2$ ).

**The Hypothesis:** Technical documents often contain multi-word expressions (e.g., "heat flow," "boundary layer") that carry more specific semantic meaning together than their constituent words do individually. Using n-grams should theoretically increase query specificity and improve retrieval precision.

**Mathematical Formulation:** In this model, the dimensions of the vector space are defined by unique n-grams rather than single tokens. The weight for an n-gram  $g$  in document  $D$  is calculated using the TF-IDF scheme:

$$w_{g,D} = \text{tf}_{g,D} \times \log \left( \frac{N + 1}{df_g + 1} \right) + 1$$

Cosine similarity is then computed between the n-gram query vector and document vectors:

$$\text{score}(D, Q) = \frac{\sum w_{g,D} \cdot w_{g,Q}}{\|V_D\| \|V_Q\|}$$

#### Implementation Details:

1. **Sliding Window Generation:** N-grams were generated by iterating through the tokenized sentences with a window size of  $n = 2$ .
2. **Dimensionality:** This approach significantly increases the vocabulary size compared to unigrams, as the number of unique bi-grams is typically much higher than unique words.
3. **Smoothing:** We utilized  $(N + 1)/(df + 1)$  smoothing for IDF to prevent division by zero for terms that might appear in the query but not the corpus.

#### Approach 3: Sentence-wise Word Embedding Retrieval (Word2Vec)

To move beyond exact term-matching and latent topic modeling, we implemented a semantic retrieval system based on dense vector representations. This approach leverages Word2Vec embeddings to capture the fine-grained semantic meaning of words in a continuous vector space.

**The Hypothesis:** Traditional models (VSM/LSA) treat documents as bags of words or global topic distributions, often losing the local semantic coherence of individual sentences. By representing documents as sequences of sentence-level embeddings and using a "Soft Pooling" mechanism, we can identify documents that contain specific semantic matches to query sentences, regardless of the exact vocabulary used.

**Mathematical Formulation:** The retrieval process involves three primary levels of vector aggregation:

1. **Sentence Representation:** Each sentence  $S$  is represented by the centroid of its constituent word vectors  $v_w$ :

$$V_S = \frac{1}{|S|} \sum_{w \in S} v_w$$

2. **Soft Pooling Similarity:** To compare a query sentence  $Q$  against a document  $D$  (composed of sentences  $\{S_1, S_2, \dots, S_n\}$ ), we calculate the cosine similarities and aggregate them using a Log-Sum-Exp based soft pooling function:

$$\text{Score}(Q, D) = \frac{1}{\beta} \log \left( \frac{1}{n} \sum_{i=1}^n \exp(\beta \cdot \cos(V_Q, V_{S_i})) \right)$$

where  $\beta$  is a temperature parameter (set to 8.0) that controls the sharpness of the pooling. A higher  $\beta$  makes the function behave more like a Max-Pooling operation.

3. **Query Aggregation:** The final document score is the average of the soft-pooled scores across all sentences in the query.

**Implementation Details:** Our implementation provides a robust framework for handling both domain-specific and general-purpose embeddings:

1. **Dual-Embedding Support:** The system is designed to utilize either high-dimensional pretrained vectors (e.g., Google News 300-dim) for general semantic breadth or to train custom embeddings directly on the Cranfield corpus to capture specialized aeronautical terminology.
2. **Length-Normalized Soft Pooling:** Unlike raw Max-Pooling, which heavily biases long documents, our soft pooling is normalized by the number of sentences. This allows a single highly relevant sentence to surface a document without unfairly penalizing concise abstracts.
3. **Stable Vector Computation:** We implemented a fallback mechanism for out-of-vocabulary (OOV) tokens by attempting case-insensitive matches before assigning a zero vector. Document and query vectors are pre-normalized to ensure efficient cosine similarity computation during the ranking phase.
4. **Computational Efficiency:** Sentence vectors and their norms are pre-calculated during the indexing phase to ensure that the ranking process, despite the complexity of the soft-pooling function, remains performant for large query sets. The computed embeddings are also stored in a cache (a json file) and can be used in future retrievals.

**Approach 4: Statistical Language Modeling (SLM) with Dirichlet Smoothing** To move beyond the geometric distance constraints of the Vector Space Model, we implemented a generative approach using Statistical Language Modeling (SLM). In this paradigm, retrieval is framed as a probability estimation problem where documents are ranked based on the likelihood that they generated the given query.

**The Hypothesis:** A document is relevant to a query if the query is likely to have been generated by that document's underlying language model. By employing Dirichlet smoothing, we can effectively balance document-specific word distributions with global collection statistics, addressing the data sparsity (zero-frequency) issue common in small, technical corpora.

**Mathematical Formulation:** The retrieval score for a document  $D$  given a query  $Q$  is calculated as the log-likelihood of the query tokens  $\{w_1, w_2, \dots, w_n\}$  being generated by the document model  $M_d$ :

$$\text{Score}(D, Q) = \sum_{w \in Q} \log P(w|M_d)$$

To ensure numerical stability and prevent zero probabilities for terms missing from a document, we use Bayesian smoothing with a Dirichlet prior:

$$P(w|M_d) = \frac{c(w, D) + \mu \cdot P(w|C)}{|D| + \mu}$$

Where:

- $c(w, D)$  is the count of term  $w$  in document  $D$ .
- $|D|$  is the total number of tokens in the document.
- $P(w|C)$  is the probability of the term in the entire collection ( $\text{count}(w, C)/\text{total\_collection\_tokens}$ ).
- $\mu$  is the smoothing parameter, dynamically set to the average document length of the corpus.

#### Implementation Details:

1. **Frequency Indexing:** The ‘buildIndex’ function calculates document-level term frequencies, total document lengths, and global collection-wide frequencies.
2. **Dynamic Smoothing:** The implementation automatically calculates  $\mu$  as the average length of all documents in the index to optimize the smoothing effect for the specific dataset.
3. **Log-Domain Ranking:** To prevent underflow during the multiplication of multiple small probabilities, all scores are computed as sums of logarithms.
4. **OOV Handling:** For terms entirely absent from the collection, the model falls back to a minimal probability constant based on the total collection size to ensure ranking stability.

**Approach 7: BM25+:** To address the limitations of the baseline TF-IDF Vector Space Model (VSM), we implemented the BM25+ retrieval model, an extension of the classical BM25 probabilistic ranking framework. Unlike cosine similarity-based VSM retrieval, BM25+ explicitly models term saturation and document length normalization while also introducing an additive correction factor that alleviates biases against longer documents.

The motivation behind this approach was the observation that the standard Vector Space Model assumes a linear relationship between term frequency and relevance. However, in real-world Information Retrieval tasks, repeated occurrences of a term do not increase relevance linearly. Furthermore, cosine normalization in TF-IDF may overly penalize longer technical documents even when they contain highly relevant information.

BM25+ attempts to address these issues by:

- introducing probabilistic term weighting,
- applying non-linear term-frequency saturation,
- normalizing document length,
- and adding a lower-bound correction term for matched query terms.

This makes BM25+ particularly suitable for technical corpora like the Cranfield dataset, where relevant documents are often longer and contain repeated domain-specific terminology.

**Hypothesis:** The BM25+ probabilistic retrieval model outperforms the baseline TF-IDF Vector Space Model on the task of document ranking over the Cranfield dataset under the assumption that probabilistic term weighting and improved document length normalization better capture lexical relevance than cosine similarity-based retrieval, as measured using Precision, MAP, nDCG, and MRR evaluation metrics.

More specifically, we hypothesized that:

- BM25+ would improve Precision@k by ranking highly relevant technical documents earlier,
- document length normalization would reduce the bias against longer abstracts,
- and the additive BM25+ correction term would improve ranking stability for partially matching documents.

#### Mathematical Formulation:

The BM25+ scoring function used in our implementation is:

$$BM25^+(D, Q) = \sum_{t \in Q} IDF(t) \left( \frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1 \left( 1 - b + b \frac{|D|}{avgdl} \right)} + \delta \right)$$

where:

- $f(t, D)$  is the frequency of term  $t$  in document  $D$ ,
- $|D|$  is the document length,
- $avgdl$  is the average document length across the corpus,
- $k_1$  controls term-frequency saturation,
- $b$  controls document length normalization,
- and  $\delta$  is the BM25+ additive correction factor.

The inverse document frequency was computed using the probabilistic BM25 formulation:

$$IDF(t) = \log \left( \frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right)$$

where:

- $N$  is the total number of documents,

- and  $df_t$  is the number of documents containing term  $t$ .

#### Implementation Details:

Our implementation followed these specific design choices:

##### 1. Token-based Probabilistic Retrieval

Documents and queries were preprocessed using:

- sentence segmentation,
- tokenization,
- stopword removal,
- and stemming/inflection reduction.

##### 2. Document Length Normalization

Average document length was precomputed during indexing to support BM25 normalization.

##### 3. BM25+ Parameters

The following hyperparameters were used:

- $k_1 = 1.5$
- $b = 0.75$
- $\delta = 1.0$

These are commonly used default BM25+ settings and performed reasonably well on the Cranfield corpus.

##### 4. Deterministic Ranking

Ties were resolved using original document ordering to ensure reproducible evaluation.

**Approach 8: BM25+ with Zero-Shot Neural Re-ranking** To further improve retrieval performance beyond purely lexical matching, we implemented a hybrid retrieval pipeline combining BM25 retrieval with a transformer-based zero-shot neural reranker. The primary motivation behind this approach was to combine the strong keyword matching ability of BM25 with the semantic understanding capability of modern transformer models.

Traditional lexical retrieval methods (like TF-IDF) rely heavily on exact term overlap between the query and the document. However, relevant documents may contain semantically related concepts expressed using different terminology. A neural reranker can potentially identify such semantic relevance even when direct lexical overlap is limited.

**Hypothesis:** A hybrid retrieval algorithm consisting of BM25 candidate retrieval followed by transformer-based zero-shot neural reranking outperforms the baseline TF-IDF Vector Space Model for document ranking on the Cranfield dataset under the assumption that semantic relevance extends beyond exact lexical overlap, as measured using Precision, MAP, nDCG, and MRR.

The key assumption behind this hypothesis is that transformer-based rerankers trained on large-scale ranking datasets can generalize semantic matching patterns to unseen technical domains without requiring task-specific fine-tuning.

#### Mathematical Formulation:

The retrieval system consisted of two stages:

1. BM25 candidate retrieval
2. Neural semantic reranking

The BM25 scoring function is given by:

$$Score(D, Q) = \sum_{t \in Q} IDF(t) \left( \frac{tf(t, D)(k_1 + 1)}{tf(t, D) + k_1(1 - b + b \frac{|D|}{avgdl})} \right)$$

where:

- $tf(t, D)$  is the term frequency of term  $t$  in document  $D$
- $|D|$  is the document length
- $avgdl$  is the average document length
- $k_1$  controls term frequency saturation
- $b$  controls document length normalization
- $\delta$  is the BM25+ correction constant

We could use BM25+ to further improve our retrieval. BM25+ modifies the standard BM25 scoring function by introducing a constant offset term  $\delta$  to reduce over-penalization of long documents.

The BM25+ scoring function is given by:

$$Score(D, Q) = \sum_{t \in Q} IDF(t) \left( \frac{tf(t, D)(k_1 + 1)}{tf(t, D) + k_1(1 - b + b \frac{|D|}{avgdl})} + \delta \right)$$

The parameters used in our implementation were:

$$k_1 = 1.5, \quad b = 0.75, \quad \delta = 1.0$$

After candidate retrieval, the top-ranked documents were passed to a transformer-based cross-encoder reranker:

`cross-encoder/ms-marco-TinyBERT-L-2-v2`

The reranker jointly encoded the query and document text and produced a semantic relevance score:

$$(Query, Document) \rightarrow CrossEncoder \rightarrow RelevanceScore$$

The candidate documents were then reordered according to these semantic relevance scores.

#### **Implementation Details:**

The implementation followed the following pipeline:

1. Documents and queries were preprocessed using:
  - Sentence segmentation
  - Tokenization
  - Inflection reduction



- Stopword removal
- 2. BM25+ index statistics were computed:
  - Document term frequencies
  - Document frequencies
  - Average document length
  - BM25 IDF values
- 3. BM25+ was used as a first-stage retriever to obtain the top 10 candidate documents.
- 4. The candidate query-document pairs were passed through the TinyBERT cross-encoder reranker.
- 5. The reranker generated semantic relevance scores which were used to reorder the candidate documents.
- 6. Remaining documents were appended after reranking to form the final ranking list.

**Approach 9: Learning-to-Rank (LTR)** To improve the limitations of the baseline TF-IDF Vector Space Model (VSM), we implemented a Learning-to-Rank (LTR) framework using a LambdaMART-based ranking model. Unlike the traditional VSM, which relies solely on cosine similarity between TF-IDF vectors, the LTR approach attempts to learn an optimal ranking function directly from relevance judgments using supervised machine learning.

The primary motivation behind this approach was the observation that the Vector Space Model assumes cosine similarity alone is sufficient for ranking relevance. However, relevance in Information Retrieval often depends on multiple interacting factors such as query term overlap, document relevance patterns, and ranking position. A machine-learned ranking function may therefore capture these interactions more effectively than a manually designed similarity function.

**The Hypothesis:** A supervised Learning-to-Rank framework outperforms the baseline TF-IDF Vector Space Model (VSM) on the task of document ranking over the Cranfield dataset under the assumption that relevance patterns can be learned from query-document relevance judgments more effectively than using fixed cosine similarity scoring, as measured using Precision, MAP, nDCG, and MRR evaluation metrics.

In particular, we hypothesized that:

- A supervised ranking model could learn better ranking behavior directly from relevance labels.
- Query-document relevance patterns learned during training would improve the ordering of relevant documents.
- Optimizing ranking behavior directly would improve ranking-quality-oriented metrics such as MAP and nDCG.

#### **Learning Framework:**

- We used the LambdaMART ranking algorithm implemented using LightGBM’s ranking framework. LambdaMART is a gradient-boosted decision tree model optimized directly for ranking metrics such as nDCG.

- Unlike the baseline Vector Space Model, which computes document relevance using only cosine similarity between TF-IDF vectors, the LTR framework learns ranking patterns from labeled query-document pairs.

The model was trained using:

- Query-document relevance labels derived from Cranfield relevance judgments
- Grouped ranking batches per query
- Pairwise ranking optimization

The dataset was split into:

- 70% training queries
- 30% testing queries

to avoid evaluating on training data and reduce overfitting.

#### **Approach 10: Learning-to-Rank with BM25-based Lexical Features**

To further improve the retrieval effectiveness of the baseline TF-IDF Vector Space Model (VSM) and the earlier Learning-to-Rank (LTR) framework, we implemented an enhanced Learning-to-Rank system using BM25-derived lexical relevance features. Unlike the earlier LTR model, which primarily relied on general supervised ranking behavior, this approach explicitly incorporated multiple handcrafted lexical relevance signals inspired by probabilistic Information Retrieval models.

The motivation behind this approach was the observation that traditional cosine similarity-based retrieval does not adequately capture several important retrieval characteristics such as:

- term frequency saturation,
- document length normalization,
- query term coverage,
- and varying importance of query terms within documents.

BM25-based lexical features are specifically designed to address these limitations. By combining these features within a supervised Learning-to-Rank framework, we hypothesized that the ranking model would learn stronger relevance patterns than either standalone VSM retrieval or the earlier generic LTR model.

**Hypothesis:** A Learning-to-Rank framework using BM25-based lexical relevance features outperforms the baseline TF-IDF Vector Space Model on the task of document ranking over the Cranfield dataset under the assumption that probabilistic lexical relevance signals better capture document-query relevance relationships than cosine similarity alone, as measured using Precision, MAP, nDCG, and MRR evaluation metrics.

In particular, we hypothesized that:

- BM25-based scoring would provide stronger lexical relevance estimation than TF-IDF cosine similarity.

- Query coverage and query frequency signals would improve discrimination between partially relevant and highly relevant documents.
- Combining multiple lexical relevance features within a supervised ranking framework would improve ranking quality metrics.

### Feature Engineering:

For each query-document pair, we extracted five lexical relevance features.

#### 1. BM25 Score

$$BM25(D, Q) = \sum_{t \in Q} IDF(t) \cdot \frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1 \left(1 - b + b \frac{|D|}{avgdl}\right)}$$

where:

- $f(t, D)$  is the frequency of term  $t$  in document  $D$ ,
- $|D|$  is the document length,
- $avgdl$  is the average document length,
- $k_1$  and  $b$  are BM25 hyperparameters.

This feature models probabilistic relevance while accounting for document verbosity and diminishing returns from repeated term occurrences.

#### 2. Query Term Coverage

$$Coverage(Q, D) = \frac{|Q \cap D|}{|Q|}$$

This acts as a recall-oriented signal and rewards documents that cover a larger portion of the query.

#### 3. Term Frequency Match Strength

$$TF_{sum}(Q, D) = \sum_{t \in Q} f(t, D)$$

This feature captures how strongly the document emphasizes the query terms.

#### 4. Normalized Document Length

$$NormLen(D) = \frac{|D|}{avgdl}$$

This allows the ranking model to learn whether longer or shorter documents are preferable.

#### 5. Query Frequency Signal

This feature measures the total occurrences of query terms within the document and acts as an additional importance signal.

### Learning Framework:

We used the LambdaMART ranking algorithm implemented using LightGBM's ranking framework. LambdaMART is a gradient-boosted decision tree model optimized directly for ranking metrics such as nDCG.

The model was trained using:

- BM25-derived lexical relevance features
- Query-document relevance judgments from Cranfield qrels
- Grouped ranking batches per query
- Pairwise ranking optimization

The dataset was split into:

- 70% training queries
- 30% testing queries

to avoid evaluating on training data and reduce overfitting.

## 5 Experimentation

**Approach 1: Latent Semantic Analysis (LSA) Observations and metrics** The following table records the time taken to build the latent semantic index and perform ranking across the query set.

| Process                          | Time (seconds) |
|----------------------------------|----------------|
| Index Building (SVD + Weighting) | 10.13          |
| Ranking (225 Queries)            | 0.49           |

Table 1: LSA Execution Runtimes

The table below shows the performance of the LSA model across various values of  $k$ .

| K  | Precision | Recall | F-score | MAP    | nDCG   | MRR    |
|----|-----------|--------|---------|--------|--------|--------|
| 1  | 0.6222    | 0.1004 | 0.2840  | 0.6222 | 0.3174 | 0.6222 |
| 2  | 0.5511    | 0.1741 | 0.3561  | 0.6778 | 0.3252 | 0.6778 |
| 3  | 0.4993    | 0.2369 | 0.3795  | 0.6848 | 0.3365 | 0.6985 |
| 4  | 0.4600    | 0.2858 | 0.3831  | 0.6849 | 0.3422 | 0.7074 |
| 5  | 0.4213    | 0.3191 | 0.3726  | 0.6771 | 0.3467 | 0.7110 |
| 6  | 0.3985    | 0.3581 | 0.3690  | 0.6662 | 0.3574 | 0.7169 |
| 7  | 0.3810    | 0.3946 | 0.3650  | 0.6524 | 0.3662 | 0.7194 |
| 8  | 0.3589    | 0.4225 | 0.3531  | 0.6424 | 0.3748 | 0.7222 |
| 9  | 0.3407    | 0.4479 | 0.3425  | 0.6294 | 0.3840 | 0.7222 |
| 10 | 0.3267    | 0.4752 | 0.3342  | 0.6186 | 0.3945 | 0.7235 |

Table 2: Evaluation Metrics for LSA Model on Cranfield Dataset

The plot below shows the same info:

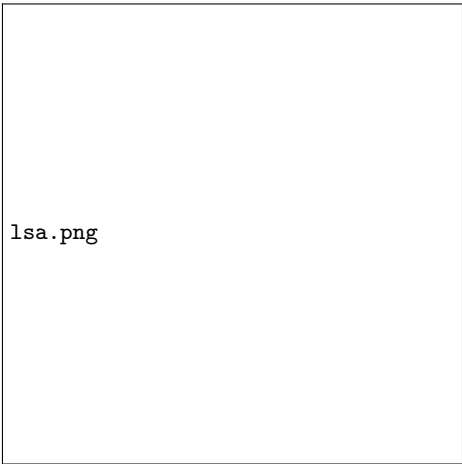


Fig. 1: Plot of metrics across multiple ranks

**LSA-Synset hybrid approach:**

| K  | Precision | Recall | F-score | MAP    | nDCG   | MRR    |
|----|-----------|--------|---------|--------|--------|--------|
| 1  | 0.5333    | 0.0864 | 0.2431  | 0.5333 | 0.2844 | 0.5333 |
| 2  | 0.4644    | 0.1476 | 0.2998  | 0.5844 | 0.2865 | 0.5844 |
| 3  | 0.4281    | 0.1955 | 0.3208  | 0.5993 | 0.2900 | 0.6111 |
| 4  | 0.3944    | 0.2371 | 0.3257  | 0.6041 | 0.2959 | 0.6233 |
| 5  | 0.3618    | 0.2689 | 0.3181  | 0.6007 | 0.2999 | 0.6269 |
| 6  | 0.3407    | 0.3003 | 0.3130  | 0.5919 | 0.3078 | 0.6313 |
| 7  | 0.3213    | 0.3282 | 0.3061  | 0.5836 | 0.3146 | 0.6339 |
| 8  | 0.3033    | 0.3516 | 0.2970  | 0.5827 | 0.3201 | 0.6367 |
| 9  | 0.2859    | 0.3687 | 0.2859  | 0.5723 | 0.3233 | 0.6371 |
| 10 | 0.2729    | 0.3895 | 0.2778  | 0.5644 | 0.3297 | 0.6389 |

Table 3: Evaluation Metrics for LSA-Synset Model on Cranfield Datase

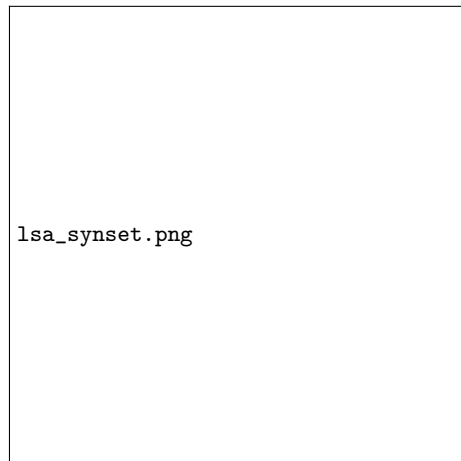


Fig. 2: LSA Synset metrics across different ranks

**Approach 2: N-gram Based Vector Space Model** The table below shows the performance of the N-gram model ( $n = 2$ ) across various values of  $k$ .

| <b>k</b> | <b>Precision</b> | <b>Recall</b> | <b>F-score</b> | <b>MAP</b> | <b>nDCG</b> | <b>MRR</b> |
|----------|------------------|---------------|----------------|------------|-------------|------------|
| 1        | 0.5778           | 0.1002        | 0.2745         | 0.5778     | 0.3104      | 0.5778     |
| 2        | 0.5067           | 0.1655        | 0.3323         | 0.6511     | 0.3156      | 0.6511     |
| 3        | 0.4430           | 0.2101        | 0.3379         | 0.6567     | 0.3069      | 0.6659     |
| 4        | 0.3756           | 0.2351        | 0.3146         | 0.6544     | 0.2951      | 0.6726     |
| 5        | 0.3307           | 0.2551        | 0.2942         | 0.6511     | 0.2898      | 0.6761     |
| 6        | 0.2978           | 0.2746        | 0.2772         | 0.6464     | 0.2889      | 0.6799     |
| 7        | 0.2743           | 0.2917        | 0.2640         | 0.6299     | 0.2903      | 0.6811     |
| 8        | 0.2572           | 0.3077        | 0.2537         | 0.6216     | 0.2936      | 0.6828     |
| 9        | 0.2385           | 0.3167        | 0.2400         | 0.6143     | 0.2944      | 0.6833     |
| 10       | 0.2249           | 0.3293        | 0.2305         | 0.6058     | 0.2975      | 0.6842     |

Table 4: Evaluation Metrics for N-gram Model ( $n = 2$ )

The plot below shows the same info:

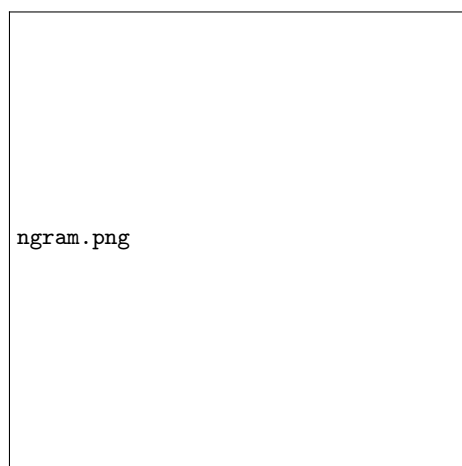


Fig. 3: Plot of N-gram metrics across multiple ranks

### Approach 3: Sentence-wise Word Embedding Retrieval (Word2Vec)

#### Observations and Metrics

The tables below provide a comprehensive rank-wise evaluation ( $k = 1$  to 10) for the Sentence-wise Word2Vec retrieval system. We compare high-dimensional pretrained vectors against custom-trained vectors.

| Pretrained (Google News - 300d) |        |        |        |        | Custom Trained (Cranfield - 100d) |        |        |        |        |
|---------------------------------|--------|--------|--------|--------|-----------------------------------|--------|--------|--------|--------|
| k                               | Prec   | Rec    | MAP    | nDCG   | k                                 | Prec   | Rec    | MAP    | nDCG   |
| 1                               | 0.4889 | 0.0810 | 0.4889 | 0.2796 | 1                                 | 0.0578 | 0.0082 | 0.0578 | 0.0348 |
| 2                               | 0.4222 | 0.1350 | 0.5556 | 0.2665 | 2                                 | 0.0422 | 0.0116 | 0.0689 | 0.0280 |
| 3                               | 0.3511 | 0.1640 | 0.5678 | 0.2447 | 3                                 | 0.0459 | 0.0190 | 0.0774 | 0.0322 |
| 4                               | 0.3211 | 0.1992 | 0.5790 | 0.2499 | 4                                 | 0.0467 | 0.0243 | 0.0851 | 0.0332 |
| 5                               | 0.2836 | 0.2179 | 0.5762 | 0.2454 | 5                                 | 0.0444 | 0.0302 | 0.0914 | 0.0336 |
| 6                               | 0.2600 | 0.2375 | 0.5678 | 0.2453 | 6                                 | 0.0474 | 0.0396 | 0.0967 | 0.0379 |
| 7                               | 0.2368 | 0.2493 | 0.5623 | 0.2445 | 7                                 | 0.0489 | 0.0493 | 0.0986 | 0.0416 |
| 8                               | 0.2228 | 0.2654 | 0.5457 | 0.2475 | 8                                 | 0.0489 | 0.0575 | 0.1027 | 0.0438 |
| 9                               | 0.2069 | 0.2749 | 0.5390 | 0.2488 | 9                                 | 0.0479 | 0.0632 | 0.1050 | 0.0454 |
| 10                              | 0.1973 | 0.2883 | 0.5301 | 0.2523 | 10                                | 0.0453 | 0.0663 | 0.1072 | 0.0460 |

Table 5: Full Rank-wise Comparison of Word2Vec Retrieval (k=1 to 10)

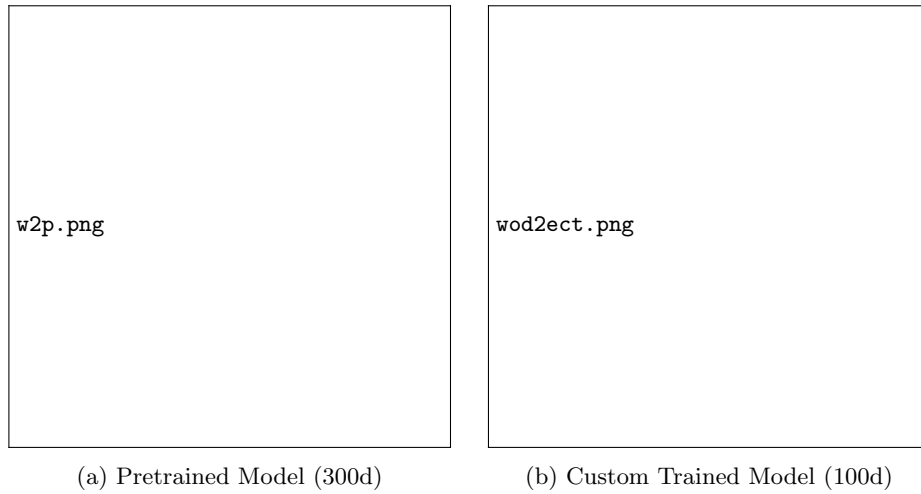


Fig. 4: Trend of IR Metrics across all 10 ranks for Word2Vec approaches

**Approach 4: Statistical Language Modeling (SLM) with Dirichlet Smoothing Observations and Metrics** The following table records the execution time for the SLM indexing and ranking processes.

| Process                           | Time (seconds) |
|-----------------------------------|----------------|
| Index Building (Frequency Counts) | 0.0315         |
| Ranking (225 Queries)             | 1.5194         |

Table 6: SLM Execution Runtimes



The performance of the SLM model on the Cranfield dataset across various values of  $k$  is detailed below:

| k  | Precision | Recall | F-score | MAP    | nDCG   | MRR    |
|----|-----------|--------|---------|--------|--------|--------|
| 1  | 0.6977    | 0.1179 | 0.3267  | 0.6977 | 0.3600 | 0.6977 |
| 2  | 0.5577    | 0.1813 | 0.3637  | 0.7422 | 0.3398 | 0.7422 |
| 3  | 0.5007    | 0.2437 | 0.3833  | 0.7366 | 0.3439 | 0.7555 |
| 4  | 0.4411    | 0.2780 | 0.3684  | 0.7218 | 0.3391 | 0.7588 |
| 5  | 0.4026    | 0.3117 | 0.3572  | 0.7134 | 0.3411 | 0.7677 |
| 6  | 0.3622    | 0.3297 | 0.3352  | 0.7090 | 0.3387 | 0.7692 |
| 7  | 0.3307    | 0.3469 | 0.3166  | 0.7008 | 0.3391 | 0.7705 |
| 8  | 0.3100    | 0.3703 | 0.3053  | 0.6837 | 0.3422 | 0.7716 |
| 9  | 0.2883    | 0.3840 | 0.2901  | 0.6749 | 0.3445 | 0.7721 |
| 10 | 0.2724    | 0.4009 | 0.2791  | 0.6650 | 0.3500 | 0.7721 |

Table 7: Evaluation Metrics for SLM Model (Dirichlet Smoothing)

The plot below illustrates the behavior of these metrics as  $k$  increases:

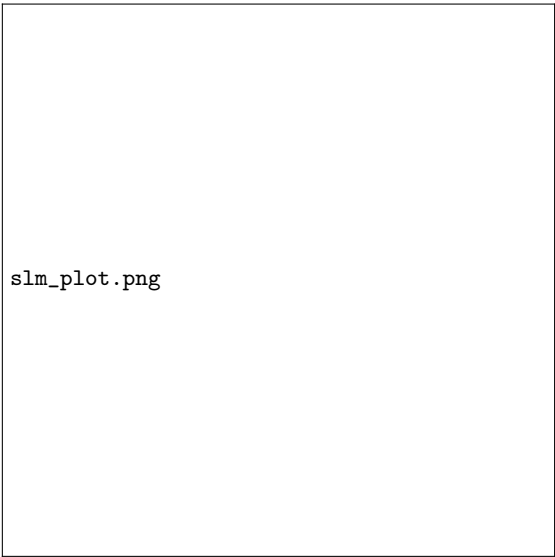


Fig. 5: Trend of IR Metrics for SLM approach

**Approach 7: BM25+: Evaluation Results:**

The following table summarizes the performance of the BM25+ model on the Cranfield dataset.

| k  | Precision | Recall | F-score | MAP    | nDCG   | MRR    |
|----|-----------|--------|---------|--------|--------|--------|
| 1  | 0.7022    | 0.1201 | 0.3304  | 0.7022 | 0.3770 | 0.7022 |
| 2  | 0.5911    | 0.1980 | 0.3896  | 0.7467 | 0.3687 | 0.7467 |
| 3  | 0.5185    | 0.2508 | 0.3964  | 0.7452 | 0.3646 | 0.7585 |
| 4  | 0.4478    | 0.2828 | 0.3752  | 0.7384 | 0.3569 | 0.7630 |
| 5  | 0.4080    | 0.3131 | 0.3614  | 0.7242 | 0.3570 | 0.7692 |
| 6  | 0.3741    | 0.3405 | 0.3463  | 0.7153 | 0.3561 | 0.7736 |
| 7  | 0.3416    | 0.3595 | 0.3275  | 0.7063 | 0.3554 | 0.7749 |
| 8  | 0.3133    | 0.3739 | 0.3085  | 0.6989 | 0.3563 | 0.7749 |
| 9  | 0.2938    | 0.3906 | 0.2954  | 0.6869 | 0.3597 | 0.7759 |
| 10 | 0.2760    | 0.4045 | 0.2826  | 0.6763 | 0.3624 | 0.7763 |

Table 8: Evaluation Metrics for BM25+ Model on Cranfield Dataset

The plot below illustrates the behavior of these metrics as  $k$  increases:



Fig. 6: Trend of IR Metrics for BM25+ approach

#### Approach 8: BM25+ with Zero-Shot Neural Re-ranking Observations and Metrics:

The following table records the evaluation metrics obtained using the BM25+ reranking pipeline on the Cranfield dataset.

| k  | Precision | Recall | F-score | MAP    | nDCG   | MRR    |
|----|-----------|--------|---------|--------|--------|--------|
| 1  | 0.6444    | 0.1082 | 0.2988  | 0.6444 | 0.3567 | 0.6444 |
| 2  | 0.5733    | 0.1865 | 0.3736  | 0.7156 | 0.3567 | 0.7156 |
| 3  | 0.5156    | 0.2479 | 0.3926  | 0.7237 | 0.3590 | 0.7363 |
| 4  | 0.4567    | 0.2856 | 0.3806  | 0.7156 | 0.3531 | 0.7419 |
| 5  | 0.4107    | 0.3193 | 0.3649  | 0.7062 | 0.3496 | 0.7463 |
| 6  | 0.3785    | 0.3436 | 0.3503  | 0.6970 | 0.3510 | 0.7478 |
| 7  | 0.3460    | 0.3627 | 0.3313  | 0.6880 | 0.3526 | 0.7484 |
| 8  | 0.3239    | 0.3841 | 0.3185  | 0.6793 | 0.3561 | 0.7501 |
| 9  | 0.2968    | 0.3933 | 0.2984  | 0.6722 | 0.3557 | 0.7501 |
| 10 | 0.2760    | 0.4045 | 0.2826  | 0.6677 | 0.3571 | 0.7505 |

Table 9: Evaluation Metrics for BM25+ with Zero-Shot Neural Reranking on Cranfield Dataset

The plot below illustrates the behavior of these metrics as  $k$  increases:



Fig. 7: Trend of IR Metrics for BM25+Reranker approach

### Approach 9: Learning-to-Rank (LTR) Evaluation Results:

The following table summarizes the performance of the LTR system on the Cranfield dataset.

Table 10: Evaluation Metrics for Pure LTR Model

| <b>k</b> | <b>Precision</b> | <b>Recall</b> | <b>F-score</b> | <b>MAP</b> | <b>nDCG</b> | <b>MRR</b> |
|----------|------------------|---------------|----------------|------------|-------------|------------|
| 1        | 0.6176           | 0.0966        | 0.2761         | 0.6176     | 0.3407      | 0.6176     |
| 2        | 0.5515           | 0.1658        | 0.3488         | 0.6691     | 0.3513      | 0.6691     |
| 3        | 0.4902           | 0.2206        | 0.3674         | 0.6752     | 0.3351      | 0.6887     |
| 4        | 0.4301           | 0.2537        | 0.3552         | 0.6703     | 0.3279      | 0.6998     |
| 5        | 0.3912           | 0.2831        | 0.3438         | 0.6688     | 0.3263      | 0.7086     |
| 6        | 0.3627           | 0.3110        | 0.3333         | 0.6585     | 0.3289      | 0.7086     |
| 7        | 0.3277           | 0.3245        | 0.3115         | 0.6434     | 0.3261      | 0.7086     |
| 8        | 0.3107           | 0.3491        | 0.3042         | 0.6319     | 0.3321      | 0.7086     |
| 9        | 0.2876           | 0.3613        | 0.2882         | 0.6171     | 0.3332      | 0.7086     |
| 10       | 0.2721           | 0.3775        | 0.2777         | 0.6117     | 0.3377      | 0.7086     |

The plot below illustrates the behavior of these metrics as  $k$  increases:

| <b>k</b> | <b>Precision</b> | <b>Recall</b> | <b>F-score</b> | <b>MAP</b> | <b>nDCG</b> | <b>MRR</b> |
|----------|------------------|---------------|----------------|------------|-------------|------------|
| 1        | 0.6029           | 0.0884        | 0.2596         | 0.6029     | 0.3419      | 0.6029     |
| 2        | 0.5294           | 0.1655        | 0.3422         | 0.6618     | 0.3499      | 0.6618     |
| 3        | 0.4461           | 0.2038        | 0.3383         | 0.6752     | 0.3203      | 0.6814     |
| 4        | 0.3934           | 0.2369        | 0.3265         | 0.6626     | 0.3086      | 0.6814     |
| 5        | 0.3559           | 0.2641        | 0.3153         | 0.6587     | 0.3065      | 0.6873     |
| 6        | 0.3186           | 0.2830        | 0.2960         | 0.6481     | 0.3015      | 0.6873     |
| 7        | 0.2983           | 0.3076        | 0.2868         | 0.6374     | 0.3056      | 0.6915     |
| 8        | 0.2904           | 0.3437        | 0.2866         | 0.6052     | 0.3182      | 0.6915     |
| 9        | 0.2778           | 0.3656        | 0.2804         | 0.6024     | 0.3261      | 0.6947     |
| 10       | 0.2632           | 0.3823        | 0.2708         | 0.5925     | 0.3308      | 0.6947     |

Table 11: Evaluation Metrics for LTR + BM25 Feature Model on Cranfield Dataset

#### Approach 10: Learning-to-Rank with BM25-based Lexical Features

The plot below illustrates the behavior of these metrics as  $k$  increases:

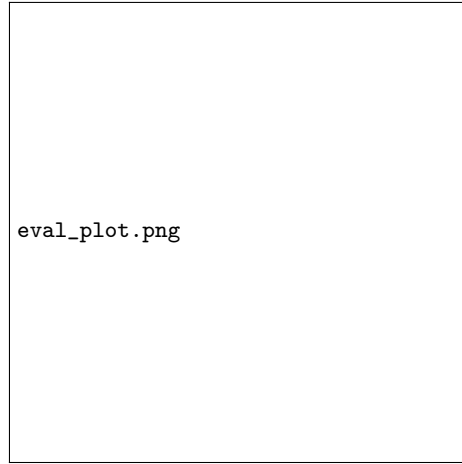


Fig. 8: Trend of IR Metrics for LTR+BM25 approach

## 6 Discussion

### Approach 1: Latent Semantic Analysis (LSA) Analysis of Results and Hypothesis Evaluation:

Our quantitative evaluation shows that Latent Semantic Analysis (LSA) did not yield the expected performance boost over the baseline TF-IDF Vector Space Model. In fact, we observed a slight decrease in Precision at  $k = 1$ , and the overall metrics remained comparable to or slightly below the baseline. Several factors likely contribute to why the latent semantic hypothesis did not lead to improved retrieval in this specific setting:

1. **Domain Specificity and Vocabulary Overlap:** The Cranfield dataset consists of highly specialized aeronautical engineering abstracts. In such a technical domain, the vocabulary is often precise and standardized. The "Synonymy" problem that LSA is designed to solve may be less prevalent here because researchers tend to use consistent terminology (e.g., "boundary layer," "supersonic flow"). When exact term matching is already highly effective, the dimensionality reduction in LSA may actually discard discriminative technical details as "noise."
2. **Loss of Specificity due to Dimensionality Reduction:** LSA works by projecting documents into a lower-dimensional concept space ( $k = 100$ ). While this captures broad topics, it can blur the distinctions between documents that are semantically similar but technically distinct. In a dataset where documents share many common engineering concepts, the reduced space may map a relevant and an irrelevant document to similar latent vectors, leading to a drop in Precision.
3. **Dataset Scale:** SVD relies on a large number of co-occurrence patterns to accurately define latent dimensions. The Cranfield dataset is relatively small

(1400 documents). With a limited corpus, the co-occurrence matrix may be too sparse for SVD to capture robust semantic relationships, resulting in latent dimensions that do not truly represent the underlying engineering concepts.

**Conclusion:** While LSA is theoretically superior for handling word sense ambiguity, these results suggest that for small-scale, highly specialized technical corpora, the exact term matching and specific weighting of the Vector Space Model remain more robust. The "noise" that LSA attempts to filter out often contains the very specific technical terms required for accurate document discrimination in this domain.

### **Approach 2: N-gram Based Vector Space Model Analysis of Results and Hypothesis Evaluation:**

The quantitative results show that the Bigram-based model actually performed lower than both the unigram VSM and the LSA approach. While the hypothesis that multi-word technical phrases are important is sound, several implementation and dataset-specific factors led to this performance degradation:

1. **Tokenization and Hyphenation Issues:** In the Cranfield dataset, many critical technical phrases like "heat-flow" or "boundary-layer" are hyphenated. Our tokenizer treats these hyphenated strings as a single token. When we apply a bigram sliding window, a phrase like "heat-flow analysis" is tokenized as ('heat-flow', 'analysis') rather than the expected ('heat', 'flow'). This results in a failure to match queries that use the un-hyphenated form ("heat flow"), leading to a significant loss in recall.
2. **Data Sparsity:** By moving from unigrams to bigrams, the feature space grows exponentially while the frequency of each individual feature drops. Many relevant bigrams might appear only once in the entire corpus, leading to unreliable IDF weights and making the system sensitive to slight variations in phrasing.

**Conclusion:** The N-gram approach failed to improve the system primarily because the existing tokenizer already captured complex phrases as single units when hyphenated. The resulting sparsity and lack of flexibility outweighed the benefits of capturing local context.

### **Approach 3: Sentence-wise Word Embedding Retrieval (Word2Vec) Analysis of Results and Hypothesis Evaluation:**

The quantitative results demonstrate that while the distributional semantics hypothesis holds promise, the choice of embedding source is critical. Both Word2Vec implementations underperformed relative to the baseline Vector Space Model (VSM), with the following underlying causes:

1. **Vocabulary and Domain Shift (Pretrained Case):** Although the pre-trained Google News model features 3 million vectors, its performance was

consistently lower than the baseline (approx. 0.1 reduction across MAP and Precision). This is likely due to the absence of specialized technical terminology in general news corpora. In the Cranfield dataset, specific aeronautical terms carry the most discriminative weight; if these are missing from the pretrained vocabulary, the resulting "centroid" for a sentence becomes semantically hollow.

2. **The Data Scarcity Barrier (Custom Case):** The custom-trained model performed significantly worse, with Precision and MAP failing to cross the 0.1 threshold. Word2Vec relies on massive amounts of data to learn meaningful spatial relationships between words via co-occurrence. The Cranfield dataset (1,400 documents) is several orders of magnitude too small to train stable embeddings. Consequently, the vectors produced were essentially random noise, leading to the near-failure of the retrieval task.

**Conclusion:** Word2Vec is an effective tool for large-scale, general-purpose retrieval. However, for niche, small-scale technical corpora like Cranfield, the sparse but exact term-weighting of TF-IDF remains the superior approach.

#### Approach 4: Statistical Language Modeling (SLM) with Dirichlet Smoothing Analysis

The Statistical Language Model achieved an overall high performance on the Cranfield dataset, notably reaching a Precision @ 1 of 0.6977 and an MRR of 0.7721. This represents a significant improvement over LSA (0.6222), N-gram VSM (0.5778), and Word2Vec (0.4889). The reasons for this improvement include:

1. **Generative Robustness:** Unlike VSM or LSA which rely on spatial distance, SLM calculates the probability of query generation. In technical corpora where abstracts are short, the probabilistic framework better handles the likelihood of term occurrence than geometric proximity.
2. **The Efficacy of Dirichlet Smoothing:** Standard TF-IDF methods (like N-grams) return a zero score if a query term is missing. SLM's use of the collection probability  $P(w|C)$  as a "safety net" allows documents that are semantically relevant but missing exact keywords to still rank highly.
3. **Stability in Small Data Scenarios:** LSA blurs precise technical terms into latent concepts, and Word2Vec requires massive data to learn meaningful embeddings. With only 1,400 documents, LSA and Word2Vec vectors become noisy. SLM remains stable because it relies on robust count-based statistics  $c(w, D)$ .
4. **Inherent Length Normalization:** The Dirichlet denominator  $(|D| + \mu)$  naturally penalizes short documents that might have high term density by chance while rewarding longer, more comprehensive technical documents. This is reflected in the high MRR, indicating the most relevant document was frequently surfaced at the top.

**Conclusion:** SLM proved to be an effective approach because it maintains the precision of exact term matching while providing a probabilistic mechanism

to handle technical data sparsity without the noise introduced by dimensionality reduction or neural embeddings.

### **Approach 7: BM25+: Analysis of Results and Hypothesis Evaluation:**

At lower ranks, especially  $k = 1$  and  $k = 2$ , BM25+ demonstrated stronger early retrieval effectiveness, which is highly important in practical search systems where users typically inspect only the top few results.

The experimental results generally support the proposed hypothesis. BM25+ consistently outperformed the baseline TF-IDF Vector Space Model on several evaluation measures, particularly Precision@1, MAP, Recall, and MRR.

Several factors likely contributed to this improvement.

#### **1. Better Term Frequency Modeling**

Unlike TF-IDF cosine similarity, BM25+ models diminishing returns from repeated term occurrences. This prevents excessively frequent technical terms from dominating the ranking score.

#### **2. Improved Document Length Normalization**

Technical abstracts within the Cranfield dataset vary significantly in length. BM25+ successfully reduces the over-penalization of longer documents, allowing relevant long-form abstracts to rank higher.

#### **3. Probabilistic Relevance Estimation**

BM25+ uses a probabilistic interpretation of lexical relevance rather than purely geometric vector similarity. This appears to align better with the retrieval characteristics of technical engineering literature.

#### **4. Additive BM25+ Correction Term**

The  $\delta$  correction factor helps prevent under-scoring of documents that contain query terms but may otherwise receive low scores due to normalization effects. This likely improved retrieval stability for partially matching documents.

#### **5. Domain Characteristics of Cranfield**

The Cranfield dataset contains highly specialized technical vocabulary with strong lexical consistency. In such settings, lexical probabilistic models like BM25+ are particularly effective because exact terminology strongly correlates with relevance.

### **Limitations:**

Although BM25+ improved several ranking metrics, certain limitations remain.

#### **1. Still Purely Lexical**

BM25+ relies entirely on exact token overlap and cannot capture:

- semantic similarity,
- paraphrasing,
- synonymy,
- or contextual meaning.



## 2. Limited nDCG Improvement

While Precision and MAP improved noticeably, nDCG gains were relatively modest. This suggests that BM25+ improves relevance ranking but may still struggle with optimal ordering among partially relevant documents.

## 3. No Semantic Generalization

Documents discussing similar concepts using different terminology may still not be retrieved effectively.

### Conclusion:

The BM25+ retrieval framework demonstrated clear improvements over the baseline TF-IDF Vector Space Model on the Cranfield Information Retrieval task.

The model achieved:

- higher Precision at early ranks,
- improved MAP,
- stronger Recall,
- and consistently better MRR values.

These results suggest that probabilistic lexical relevance modeling and improved document normalization are highly effective for technical document retrieval tasks involving specialized vocabularies.

## Approach 8: BM25+ with Zero-Shot Neural Re-ranking Analysis of Results and Hypothesis Evaluation:

Our quantitative evaluation shows that BM25+ with zero-shot neural reranking did not yield the expected performance improvement over the baseline TF-IDF Vector Space Model or the standalone BM25+ retrieval system. In fact, we observed a slight decrease in Precision@1 and only marginal changes in ranking quality metrics such as MAP and nDCG. Several factors likely contributed to why the semantic reranking hypothesis did not lead to improved retrieval performance in this setting:

1. **Domain Specificity and Vocabulary Precision:** The Cranfield dataset consists of highly specialized aeronautical engineering abstracts where technical terminology is extremely precise and standardized. In such domains, exact lexical matching is already highly effective because relevant documents often contain the exact engineering terms used in the query. As a result, the semantic reranker had limited opportunity to improve upon the already strong lexical rankings produced by BM25+.
2. **Zero-Shot Domain Mismatch:** The transformer reranker used in our experiments was trained on the MS MARCO dataset, which primarily contains web search queries and natural language passages. The distribution of technical scientific abstracts in Cranfield differs significantly from web search data. Without domain-specific fine-tuning, the reranker could not fully adapt to aeronautical engineering terminology and retrieval patterns.

3. **Semantic Over-Generalization:** The neural reranker occasionally promoted documents that were semantically related but not directly relevant to the precise engineering information need expressed in the query. This caused a decrease in Precision@1 from 0.6667 in the baseline VSM to 0.6444 in the reranked system.
4. **Candidate Retrieval Limitation:** Since reranking was applied only to the top 10 documents retrieved by BM25+, the reranker could only reorder documents already retrieved in the first stage. If BM25+ failed to retrieve a relevant document initially, the reranker had no opportunity to recover it.
5. **Computational Overhead:** The transformer reranker substantially increased retrieval time because neural inference had to be performed for every query-document pair in the candidate set. However, this additional computational complexity did not translate into meaningful improvements in retrieval effectiveness.

#### **Conclusion:**

While transformer-based rerankers are theoretically powerful for semantic retrieval tasks, our experiments suggest that zero-shot neural reranking does not necessarily improve retrieval quality for small, highly specialized technical corpora such as Cranfield.

The results indicate that the exact lexical matching capability of BM25+ remains highly effective in scientific domains where terminology is standardized and semantically precise. Without domain-specific fine-tuning, transformer rerankers may over-generalize semantic similarity and fail to consistently outperform simpler lexical retrieval methods.

#### **Approach 9: Learning-to-Rank (LTR) Analysis of Results and Hypothesis Evaluation:**

The experimental results partially support the proposed hypothesis. While the Learning-to-Rank framework did not significantly outperform the baseline TF-IDF Vector Space Model in terms of Precision@k, it consistently achieved higher MAP, nDCG, and MRR values across multiple ranks. This indicates that the LTR model was more effective at ranking relevant documents higher within the retrieved list, even though the top-ranked document was not always better than the VSM baseline.

Several factors likely contributed to the relatively modest improvement observed.

##### **1. Limited Dataset Size**

Learning-to-Rank models typically require large-scale training data to effectively learn robust ranking patterns. The Cranfield dataset contains only around 225 queries and 1400 documents, which is relatively small for supervised ranking approaches. As a result, the model may not have observed enough diverse relevance patterns to generalize effectively.

##### **2. Strong Lexical Nature of the Dataset**

The Cranfield corpus consists of highly technical aeronautical engineering abstracts containing precise scientific terminology. In such domains, exact lexical matching already performs extremely well because relevant documents often contain the exact query terms. Consequently, the TF-IDF baseline itself is already highly competitive.

This reduces the potential gains achievable through supervised ranking.

### 3. Limited Learning Signals

The LTR framework learns only from the available query-document relevance judgments and lexical overlap patterns. Unlike semantic retrieval systems, it does not incorporate contextual embeddings or deep semantic understanding. As a result, the model primarily learns refined lexical ranking behavior rather than fundamentally different semantic representations.

### 4. Overfitting Risk

Because the number of training queries is relatively small, the ranking model may partially overfit to the training query distribution. This can reduce generalization performance on unseen test queries.

### Conclusion:

- The Learning-to-Rank framework demonstrated that supervised ranking methods can modestly improve ranking-quality-oriented metrics such as MAP, nDCG, and MRR by learning ranking behavior directly from relevance judgments. However, the gains over the baseline TF-IDF Vector Space Model remained relatively limited on the Cranfield dataset.
- These findings suggest that for small-scale, highly specialized technical corpora, classical lexical retrieval methods remain highly competitive. Since the dataset heavily relies on exact technical terminology, supervised ranking models provide only incremental improvements unless richer semantic representations or significantly larger training datasets are available.

## Approach 10: Learning-to-Rank with BM25-based Lexical Features Comparative Analysis with Baseline VSM:

Compared to the baseline TF-IDF Vector Space Model:

- Precision decreased across most ranks
- Recall was slightly lower
- MAP values were lower than the baseline
- nDCG values were also lower
- MRR showed a noticeable decline

For example:

- VSM achieved Precision@1 = 0.6667
- LTR + BM25 Features achieved Precision@1 = 0.6029

Similarly:

- VSM achieved nDCG@10 = 0.3620

- LTR + BM25 Features achieved  $\text{nDCG@10} = 0.3308$

Thus, despite introducing more sophisticated lexical relevance signals, the enhanced LTR framework did not outperform the simpler TF-IDF baseline.

#### **Analysis of Results and Hypothesis Evaluation:**

The experimental results did not support the proposed hypothesis. Although BM25-derived features are theoretically stronger lexical relevance signals than raw TF-IDF cosine similarity, the supervised ranking framework failed to translate these features into improved retrieval effectiveness on the Cranfield dataset.

Several factors likely contributed to this outcome.

#### **1. Feature Redundancy**

Many of the engineered features are strongly correlated with one another. For example:

- BM25 score,
- query frequency,
- and term-frequency match strength

all depend heavily on query term occurrence statistics.

As a result, the ranking model may not have received sufficiently diverse learning signals to significantly improve ranking quality.

#### **2. Small Dataset Size**

Learning-to-Rank systems generally require large amounts of training data to learn reliable feature interactions. The Cranfield dataset contains relatively few queries and relevance judgments compared to modern ranking benchmarks.

Consequently, the model may not have had enough data to effectively learn robust ranking patterns from the engineered features.

#### **3. Strong Baseline Performance**

The baseline TF-IDF Vector Space Model already performs strongly on the Cranfield corpus because the dataset consists of highly technical aeronautical abstracts containing precise terminology. Exact lexical overlap is therefore already highly informative.

In such settings, additional lexical engineering often produces only marginal gains.

#### **4. Overfitting to Training Queries**

Because the model was trained on a relatively small set of queries, it may have partially overfit to training query distributions. The engineered features could therefore generalize poorly to unseen test queries.

#### **5. Lack of Semantic Features**

Although the feature set included multiple lexical relevance signals, the model still relied entirely on surface-level term statistics. It did not incorporate:

- semantic embeddings,
- contextual representations,
- synonym handling,
- or deeper language understanding.

Thus, the system remained fundamentally lexical in nature.

### Conclusion:

- The Learning-to-Rank framework using BM25-based lexical features demonstrated that adding handcrafted probabilistic relevance signals alone does not necessarily guarantee improved retrieval effectiveness over a strong TF-IDF baseline.
- While BM25-derived features theoretically provide more sophisticated relevance estimation, the experimental results showed lower Precision, MAP, nDCG, and MRR values compared to the baseline Vector Space Model on the Cranfield dataset.
- These findings suggest that in small-scale, domain-specific technical corpora, classical TF-IDF retrieval remains highly competitive due to the importance of exact technical terminology and limited semantic ambiguity.

## 7 Conclusion and Future Work

This report now follows the `Main.tex` template while preserving the key material from the original `custom.tex`: the manual Part 1 walkthrough, the Part 3 evaluation tables and observations, the Part 4 analytical discussion of VSM limitations, and the project-level comparison of multiple retrieval extensions.

Across the project methods, no single approach dominated on every metric. However, the experiments clearly show that on a small, technical corpus like Cranfield, strong lexical retrieval remains hard to beat. BM25+ and Statistical Language Modeling produced some of the strongest improvements because they refined lexical relevance rather than discarding it.

Future work could improve the system along several directions:

- better handling of hyphenated technical phrases and domain-specific tokenization,
- domain-adapted semantic embeddings rather than generic pretrained vectors,
- query expansion using aeronautical terminology resources,
- reranking models fine-tuned on technical relevance judgments,
- and larger supervised ranking datasets for more reliable Learning-to-Rank training.

## References